

**Faculty of Information and Communication Technology**

Diploma: Computer Systems Engineering

Final Project Report**Facial Recognition Door Lock System**

Module: Project Design 376

Module code: PD2316D

Student name	Nakedi Tumelo Peta
Student number	223570984
Email	makgwatatumelo46@gmail.com
Lecturers	Muwanguzi Mark Ntume and Chunling Du
Submission date	25 October 2025

Contents

Contents	2
1. Abstract.....	3
2. Introduction	3
3. Background.....	4
4. Past and Related Work	5
4.1 Comparison	5
5. Project Management	6
5.1 Management Strategy	6
5.2 Task Breakdown and Contribution	6
5.3 Project Plan	7
6. Technical Design	7
6.1 System Architecture	7
6.2 Hardware Components.....	8
6.3 Wiring and Connections.....	8
6.4 Power Calculations	9
6.5 Circuit Diagram.....	10
6.6 System Architecture Overview	10
7. Results and Analysis.....	11
7.1 Main Results.....	11
7.2 Verification Accuracy and Tolerance.....	12
7.3 Latency and Responsiveness	12
7.4 Comparison with Existing Approaches.....	13
8. Challenges Faced	13
9. Final Budget.....	15
10. Project Deliverables	16
10.1 Comparison with Initial Goals	16
11. Recommendations and Future Work.....	17
12. Conclusions	18
13. References.....	19
14. Appendix A: Project Progression Images	20
15. Appendix B: Source Code	23

1. Abstract

Traditional access control methods such as keys and access cards can be lost, stolen, or copied, and managing them across several users adds overhead. This project builds a prototype door lock that uses facial recognition instead, so a person's own face becomes the credential. The system combines a standard PC webcam, a Python Flask web application, Google Firebase for authentication and data storage, and an ESP32 microcontroller that drives the physical lock.

The workflow is straightforward. A user registers through a web page, and the system captures a photo of their face, converts it into a numerical embedding with a face recognition library, and stores that embedding in a Firebase Realtime Database. To log in, the user presents their face again. The new image is captured, converted to an embedding, and compared against the stored one using a tolerance threshold. When the match succeeds, the Flask server sends an HTTP command over Wi-Fi to the ESP32, which energises a relay to open the solenoid lock and lights a green LED. A failed match keeps the lock closed and lights a red LED. Every attempt, successful or not, is logged with a timestamp so access can be audited later. The result is a working link between a web application and IoT hardware, with an adjustable matching tolerance that lets the operator trade strictness against convenience.

2. Introduction

Keys, ID cards, and PIN codes have run access control for a long time, but they share the same weaknesses: they can be lost, stolen, forgotten, or copied, and they create administrative work whenever credentials change. Biometric authentication avoids most of these problems because it ties access to something the user already carries, their own physical features. Facial recognition fits well here. It does not require the user to touch a sensor or remember anything, and a face is hard to copy or hand over.

Most off-the-shelf facial recognition locks are expensive and hard to extend. They tend to ship as closed units that do not integrate easily with custom hardware or cloud services. That leaves room for a system that is affordable, open to modification, and that ties together real-time recognition, cloud-based user management, and physical device control.

This project sets out to build that system as a working prototype. A few problems had to be solved along the way: keeping recognition accurate across different lighting and angles, keeping the delay between verification and the lock responding short enough to feel immediate, protecting biometric data in transit and at rest, and getting four very different technologies (an ESP32 firmware, Firebase, a JavaScript frontend, and a Python Flask backend) to work together cleanly.

The system uses a distributed design. A webcam captures the user's face in the browser, a Flask server turns that image into a facial embedding using a face recognition library, and Firebase handles authentication and stores user data. Once a face is verified, Flask sends an HTTP command over Wi-Fi to an ESP32, which opens the lock and shows the result on coloured LEDs.

The prototype assumes a few things about its environment: a modern browser with a working webcam, the ESP32 and the server on the same Wi-Fi network, reasonable lighting

for recognition, and a correctly configured Firebase project. Within those bounds, the objectives were to let users register with a face capture, log in and verify against the stored face, pass results between Flask and the ESP32 in real time, and log both successful and failed attempts in Firebase. The coloured LEDs and the solenoid lock together stand in for a real door lock and confirm that the system works from end to end.

3. Background

Controlling who can enter a physical space is a basic security requirement for homes, businesses, and restricted areas. For decades this has been handled with physical tokens like mechanical keys, key cards, and fobs, or with knowledge-based methods like PIN codes. All of these have built-in weaknesses. PINs can be forgotten or observed; cards and keys can be lost, stolen, or duplicated; and keeping track of credentials for several users takes ongoing administration.

Biometrics take a different approach by authenticating people based on their own biological or behavioural traits. Common examples include fingerprint scanning, facial recognition, iris scanning, and voice recognition. Because these traits belong to the person, they are much harder to lose, steal, or forge than a card or a password. The user effectively becomes the credential, which can improve both security and convenience.

Facial recognition identifies or verifies a person by analysing features taken from their face. Most modern systems work in three stages:

- **Face detection:** locating one or more faces within an image or video frame.
- **Feature extraction:** measuring distinctive features of the detected face, such as the distances between the eyes and the shape of the nose and jaw, and converting them into a compact set of numbers. This numerical representation is usually called a face embedding or template, and it is a mathematical signature of the face. Storing the embedding is safer and more private than storing the actual photograph.
- **Comparison and matching:** comparing a freshly generated embedding from the live camera against one or more stored embeddings, usually by calculating the mathematical distance between them. If that distance falls below a set threshold, the faces are treated as a match.

The Internet of Things (IoT) connects physical devices to a network so they can send and receive data and commands. For access control, this means a locking mechanism can be operated remotely in response to a decision made elsewhere, typically on a local or cloud-based server. Microcontrollers like the ESP32 are popular for this because they have built-in Wi-Fi, cost very little, and have enough processing power to handle network traffic and drive connected hardware such as sensors, relays, and LEDs.

This project brings these ideas together. A Flask server written in Python manages users and runs facial recognition through a face recognition library. Firebase provides cloud authentication using email and password, and a Realtime Database stores user profiles and the face embeddings. The Flask server and the ESP32 talk to each other over the local Wi-Fi network using simple HTTP requests, so the web application can trigger a physical action, such as opening the lock, based on the result of a verification.

4. Past and Related Work

Facial recognition for access control is a well-developed field with a range of existing solutions, spanning commercial products, do-it-yourself builds, and cloud services. Looking at how they work helps put this project's approach in context.

Commercial systems usually combine the camera, processing unit, and lock interface into a single integrated unit. They tend to perform well and come with vendor support, but they can be expensive, offer little room for customisation, and rely on proprietary algorithms that are hard to integrate with other platforms.

Open-source and hobbyist projects often use a single-board computer such as a Raspberry Pi paired with a camera module, running detection and recognition locally with libraries like OpenCV or Dlib. These builds are highly customisable, but they can be fiddly to set up, they assume some Linux administration knowledge, and they may struggle with real-time performance on lower-powered hardware. Web-based management and cloud storage, if present at all, are usually bolted on separately. Some projects go even smaller with ESP32-CAM modules for self-contained recognition, though hardware limits mean these can only handle a small number of faces with simpler models.

A third common approach uses cloud facial recognition APIs such as AWS Rekognition, Azure Face API, or Google Cloud Vision. These offload the heavy feature extraction and matching to powerful cloud servers and give access to strong pre-trained models. The trade-off is that facial data has to leave the device, which usually means a constant internet connection, ongoing usage costs, and added privacy concerns. Something local is still needed to take the cloud's answer and actually control the hardware.

This project takes a middle path by combining affordable, easily available components in a distributed design aimed at a home or small office. A few things make the approach distinct:

- **A hybrid split of work.** The browser handles initial face detection, guidance, and cropping with JavaScript (`face-api.js`), while a Python Flask backend does the heavier embedding comparison with the face recognition library. This uses the PC's processing power without needing dedicated edge-AI hardware.
- **Web and IoT working together.** Everyday web technologies (Flask, HTML, JavaScript) drive an ESP32 over Wi-Fi, so a verification in the browser produces a real physical response on the hardware through ordinary local network requests.
- **Accessible parts.** The build uses common hardware (a PC, a webcam, an ESP32) and mostly open-source software, plus Firebase's free tier for authentication and storage, which keeps it cheap and easy to reproduce compared with commercial units or systems that need AI accelerators.
- **Practical extras.** Beyond basic verification, the system logs both successful and failed attempts, which matters for security auditing, and exposes an adjustable tolerance so the balance between security and convenience can be tuned. Cropping the face in the browser before upload also reduces the work the backend has to do.

4.1 Comparison

Compared with a bare ESP32-CAM or a simple Raspberry Pi build, this system adds proper web-based user management, cloud-stored embeddings, explicit logging of failed attempts,

and guided face capture in the browser. Compared with a pure cloud API, it keeps the core matching logic local inside the Flask app and uses a simpler local method to control the hardware.

On performance, running the final comparison on the PC's CPU against a pre-cropped image generally gives better accuracy and speed than running full models on a low-power device. The main source of delay is the local network hop between Flask and the ESP32. The system does not match the anti-spoofing or throughput of specialised commercial units.

The overall focus is integration: a practical blueprint for tying together accessible web and IoT technologies without needing deep embedded-AI expertise or costly hardware. Security rests on standard Firebase authentication and the difficulty of spoofing the embeddings, tunable through the tolerance value, rather than on dedicated liveness detection.

5. Project Management

5.1 Management Strategy

The project followed an iterative approach. Because the prototype pulled together a web application, backend logic, cloud services, and hardware, building it in cycles gave room to adjust as problems surfaced. The core functions were implemented first, then tested and refined, after which more features such as hardware communication, error handling, and a better interface were added. Working this way meant issues like unreliable face detection and ESP32 connectivity showed up early, while there was still time to change course, rather than being locked into an upfront design. The resources used were modest: an ESP32, a webcam, a standard development PC, basic electronic components such as LEDs and resistors, and Firebase's free cloud services.

5.2 Task Breakdown and Contribution

This was a solo project, so I handled every part of it: design, programming, testing, and documentation. The main tasks were:

- **Environment setup:** configuring the Arduino IDE for the ESP32, creating the Python environment (myenv38), installing the required libraries (Flask, Pyrebase, face recognition, requests), and setting up the Firebase project with Authentication and the Realtime Database.
- **Flask backend:** building registration, login, and logout on top of Firebase Authentication; adding routes (/capture, /verify) to handle image uploads from the browser; generating and comparing embeddings with the face recognition library; reading and writing user data and embeddings in the Realtime Database; sending HTTP commands to the ESP32; managing user sessions; and logging both successful and failed access attempts.
- **Frontend:** building the Login, Register, Capture, Verify, and Dashboard pages as HTML templates styled with Tailwind CSS, with client-side JavaScript for webcam access (getUserMedia), real-time detection and guidance (face-api.js), cropping, and asynchronous calls to the backend (fetch).
- **Hardware integration:** wiring the ESP32 to the indicator LEDs and the relay, writing the Arduino firmware to join Wi-Fi and run a small web server, and controlling the GPIO pins based on the commands received from Flask.

- **Testing and debugging:** checking each piece on its own, the backend routes, the frontend interactions, ESP32 connectivity, and Firebase operations, and tracking down problems such as JavaScript errors, Python logic bugs, network issues, and incorrect match tolerances.
- **Documentation:** writing this report, commenting the code, and recording how the system is set up and how it fits together.

The work drew on web development (HTML, CSS, JavaScript), cloud database administration (Firebase), embedded programming (Arduino and C++ for the ESP32), and Python with Flask.

5.3 Project Plan

The schedule below shows how the work was sequenced across the project, from environment setup and the Flask backend through the frontend, hardware integration, logging, and final report writing.

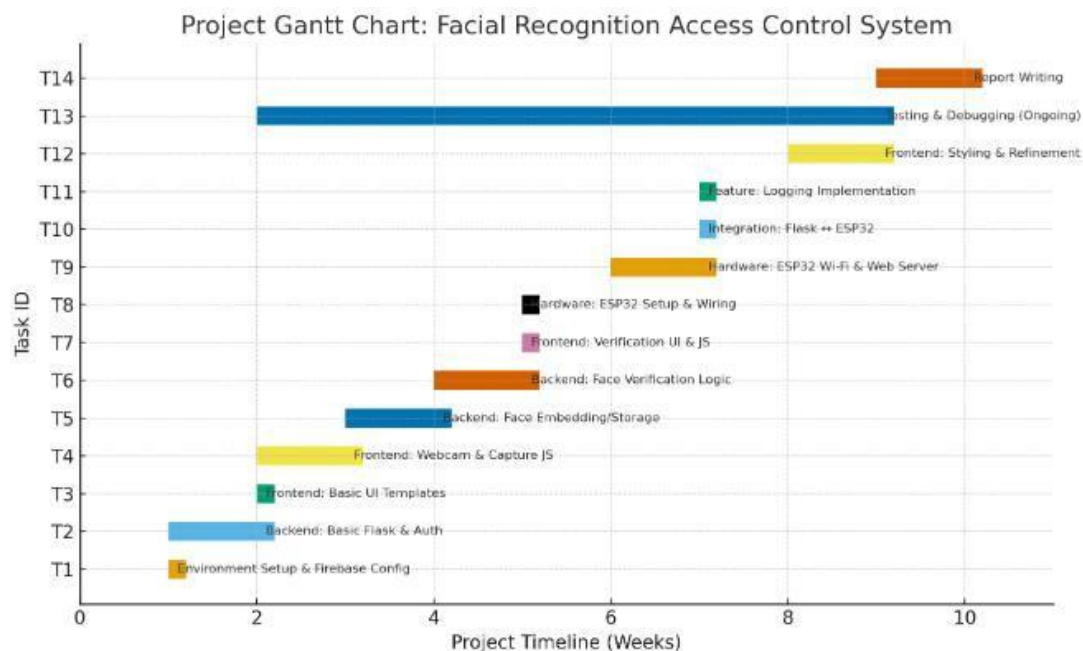


Figure 1. Project Gantt chart.

6. Technical Design

6.1 System Architecture

The system uses a distributed architecture with four main parts: a web frontend, a Flask backend server, the Firebase cloud backend, and an ESP32 microcontroller that acts as the hardware actuator. They interact as follows.

The web frontend runs in the user's browser. It shows the pages (Login, Register, Capture and Verify, Dashboard), takes user input, accesses the PC's webcam through browser APIs, does the initial face detection and cropping with face-api.js, and talks to the Flask backend through asynchronous HTTP requests.

The Flask backend is a Python application. It forwards login credentials to Firebase Authentication, takes the cropped face images, generates and compares embeddings with

the face recognition library, reads and writes user data and embeddings in the Realtime Database, logs access attempts, and sends commands to the ESP32 over the local network. It keeps track of who is logged in using Flask's session management.

Firebase provides two cloud services. Firebase Authentication manages user accounts through email and password sign-up and sign-in. The Firebase Realtime Database, a NoSQL cloud database, stores each user's profile (first name, last name, email), their face embedding, and timestamped logs of successful (`access_logs`) and failed (`failed_logs`) attempts, all organised under a unique user ID.

The ESP32 is a low-cost Wi-Fi microcontroller programmed with the Arduino framework. It joins the local network, runs a small web server, and listens for specific HTTP requests (`/access_granted` or `/access_denied`) from the backend. When a command arrives, it drives the GPIO pins connected to the indicator LEDs (green for granted, red for denied or idle) and the relay that controls the physical lock.

6.2 Hardware Components

The prototype was built from the following parts:

- ESP32 WROOM-32E development board
- I2C LCD
- Green LED (5 mm)
- Red LED (5 mm)
- Two 220 Ohm resistors (current limiting for the LEDs)
- 1 kOhm resistor (relay drive)
- 5 V relay module
- 12 V solenoid lock
- 12 V battery
- TIP41C transistor
- Buzzer
- PVC pipe and door hinges (enclosure)
- Veroboard and jumper wires
- Micro-USB cable (power and programming)
- Webcam
- Spray paint, glue, nails, nuts and bolts

6.3 Wiring and Connections

The build uses two power supplies: 5 V for the ESP32 and 12 V for the solenoid lock. The wiring is spread across two Veroboards with a shared ground.

Veroboard 1

- ESP32 GND to the GND rail of Veroboard 1.
- 5 V GPIO pin of the ESP32 to VCC of the relay module.
- GPIO 27 to the TIP41C base through the 1 kOhm resistor.

- TIP41C collector to the IN of the relay.
- TIP41C emitter to the GND rail of Veroboard 1.
- GPIO 26 to the anode of the green LED; cathode through a 220 Ohm resistor to the buzzer and the GND rail.
- GPIO 25 to the anode of the red LED; cathode through a 220 Ohm resistor to the GND rail.
- Relay GND to the GND rail of Veroboard 1.
- ESP32 supplied with 5 V.

Veroboard 2

- GND rail of Veroboard 1 tied to the GND rail of Veroboard 2 for a common ground.
- Relay COM to VCC of Veroboard 2.
- Relay NO to the red wire of the solenoid lock.
- Black wire of the solenoid lock to the GND rail of Veroboard 2.
- 12 V supplied to Veroboard 2 for the solenoid lock.

I2C LCD

- VCC to 3.3 V on the ESP32.
- GND to GND on the ESP32.
- SDA to GPIO 21.
- SCL to GPIO 22.

6.4 Power Calculations

The two governing equations are Ohm's law, $V = I \times R$, and power, $P = V \times I$. The calculations below estimate the current and power for each part of the circuit and then the total for the full setup.

LED circuit (single LED)

Source voltage	$V_s = 3.3 \text{ V}$	(ESP32 GPIO)
LED forward voltage	$V_f = 2.12 \text{ V}$	
Resistor	$R = 220 \text{ Ohm}$	
Voltage across R	$V_R = V_s - V_f = 3.3 - 2.12$	$= 1.18 \text{ V}$
LED current	$I = V_R / R = 1.18 / 220$	$= 5.36 \text{ mA}$
Resistor power	$P_R = V_R \times I = 1.18 \times 5.36 \text{ mA}$	$= 6.33 \text{ mW}$
LED power	$P_L = V_f \times I = 2.12 \times 5.36 \text{ mA}$	$= 11.36 \text{ mW}$
Total per LED		$= 17.7 \text{ mW}$

Relay drive (ESP32 to 1 kOhm to TIP41C base)

$V_{BE} \sim 0.7 \text{ V}$ for the TIP41C
 Base current $I_{base} = (3.3 - V_{BE}) / 1000 = (3.3 - 0.7) / 1000 = 2.6 \text{ mA}$

Relay coil (via TIP41C)

$P_{relay} = V \times I = 5 \text{ V} \times 0.08 \text{ A} = 0.4 \text{ W}$

Solenoid lock

$P = V \times I = 12 \text{ V} \times 0.5 \text{ A} = 6 \text{ W}$

ESP32

$I = 150 \text{ mA}$
 $P = V \times I = 5 \text{ V} \times 0.15 \text{ A} = 0.75 \text{ W}$

LCD

$V = 3.3 \text{ V}$

$$I = 50 \text{ mA}$$

$$P = V \times I = 3.3 \text{ V} \times 0.05 \text{ A} = 0.165 \text{ W (165 mW)}$$

Total system power (full setup)

$$P_{\text{total}} = 0.0177 + 0.0086 + 0.4 + 6 + 0.75 + 0.165$$

(LEDs) (relay) (coil) (lock) (ESP32) (LCD)

$$P_{\text{total}} = 7.3 \text{ W}$$

6.5 Circuit Diagram

The circuit below shows the ESP32, the two indicator LEDs with their 220 Ohm resistors, the 5 V relay module driven through the TIP41C, the 12 V supply, and the solenoid lock.

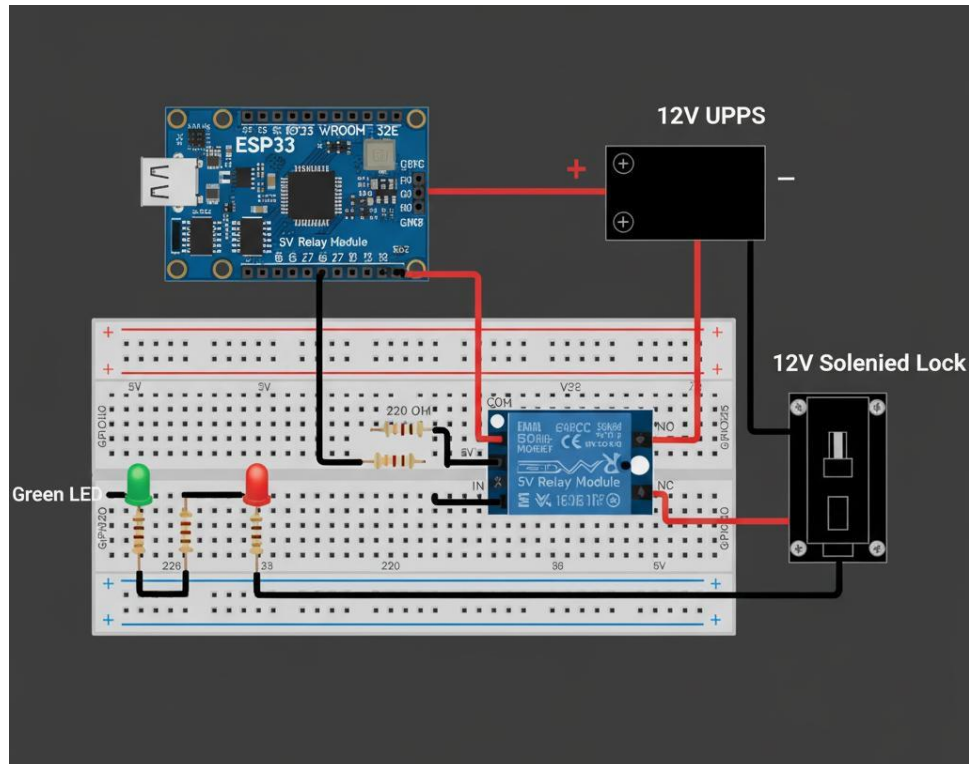


Figure 2. Circuit and breadboard layout.

6.6 System Architecture Overview

The diagram below traces a request through the whole system, from capturing or verifying an image in the browser, to authentication and storage in Firebase, to the verification result driving the ESP32 and the lock.

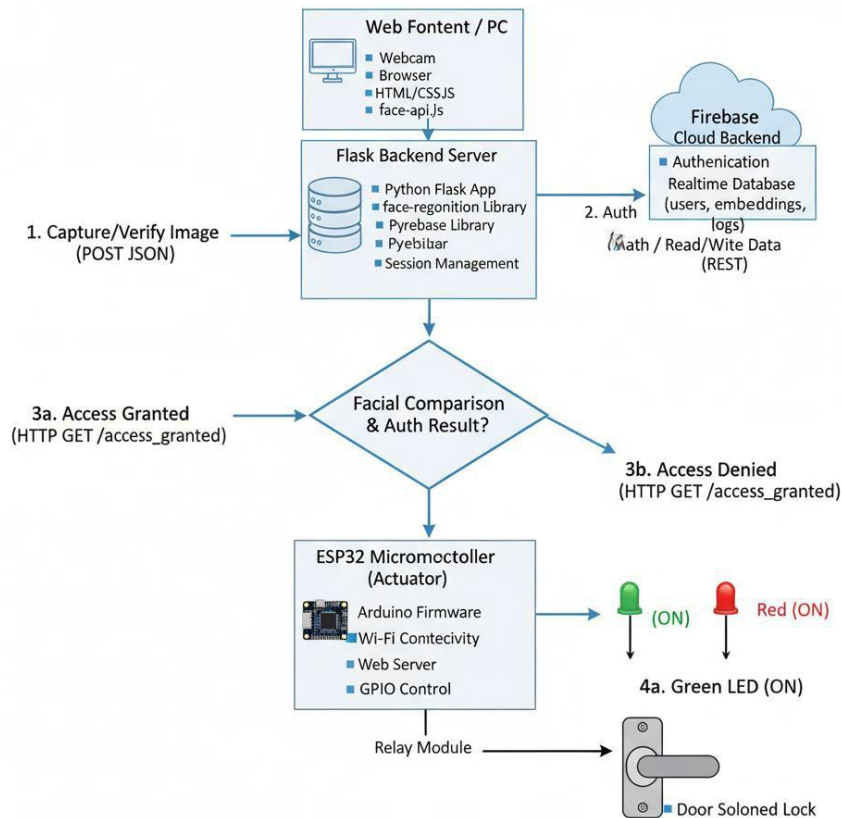


Figure 3. System architecture and data flow.

7. Results and Analysis

7.1 Main Results

The project produced a working prototype that demonstrated the full workflow from end to end:

- **User management:** users can enter their first and last names and create an account with an email and password through the web interface.
- **Face capture:** the system reliably captures a face through the capture page using an ordinary webcam. face-api.js helps the user centre their face, crops the detected face, and sends it to the Flask backend.
- **Embedding storage:** the backend generates a 128-dimensional embedding from the cropped image with the face recognition library and stores it in the Realtime Database against the registered user.
- **Face verification:** during verification, the system captures and crops a face, sends it to the backend, generates an embedding, retrieves the stored embedding from Firebase, and compares them with `face_recognition.compare_faces()`.
- **Hardware actuation:** depending on the result, the backend sends the matching command (`/access_granted` or `/access_denied`) to the ESP32 over Wi-Fi. On a match, the relay energises and the solenoid lock opens; the ESP32 lights the green LED for granted and the red LED for denied.

- **Access logging:** successful and failed attempts are stored under the user's record in Firebase, each timestamped in South African Standard Time (SAST), and the dashboard shows the retrieved logs.
- **Interface:** the pages are styled with Tailwind CSS for a clean, responsive, and consistent look across registration, login, capture, verification, and the dashboard.

7.2 Verification Accuracy and Tolerance

How well the system accepts the right person while rejecting everyone else depends heavily on the tolerance value in `face_recognition.compare_faces()`, which sets the largest Euclidean distance between two embeddings that still counts as a match. A lower tolerance means stricter matching. Testing used the registered user and one or more unregistered people under steady indoor lighting. The table below shows the results across different tolerance values in `app.py`.

Tolerance	Registered accepted (true positive)	Registered rejected (false rejection)	Unregistered accepted (false acceptance)	Outcome
0.6 (default)	20 / 20 (100%)	0%	2 / 20 (10%)	Too lenient; security risk
0.5 (selected)	19 / 20 (95%)	5%	0 / 20 (0%)	Best balance
0.45	17 / 20 (85%)	15%	0 / 20 (0%)	Stricter; more false rejections
0.4	15 / 20 (75%)	25%	0 / 20 (0%)	Too strict to be usable

At the default tolerance of 0.6, matching was too loose and let an unregistered user through 10% of the time, which is a real security problem. Lowering the tolerance to 0.5 gave the best balance: it rejected every unregistered attempt while only once turning away the registered user, most likely because of a small change in pose or expression compared with the single registration photo. Tightening further to 0.45 and then 0.4 kept security high but pushed false rejections up to 15% and then 25%, which makes the system frustrating for the authorised user. The chosen value of 0.5 keeps false acceptances at zero while still letting the registered user in about 95% of the time. Cropping the face in the browser before sending it to the backend likely helps this reliability by giving the backend a consistent, centred image. Even so, large changes in lighting, angle, or expression can still cause a false rejection.

7.3 Latency and Responsiveness

Informal timing measured how long it took the matching LED on the ESP32 to light up after the Verify Identity button was pressed:

- **Browser processing (`face-api.js` crop):** under 100 ms, effectively negligible.
- **Network, browser to Flask:** variable, usually 50 to 200 ms on local Wi-Fi.
- **Flask backend recognition:** roughly 200 to 500 ms, mostly depending on the server PC's CPU. This covers decoding the image, generating the embedding, fetching the stored embedding from Firebase, and comparing them.

- **Network, Flask to ESP32:** variable, usually 30 to 100 ms on local Wi-Fi.
- **ESP32 processing and actuation:** under 50 ms, negligible.

Under good local network conditions the total comes to roughly 300 to 850 ms, which is acceptable for access control. The backend comparison and the network hops are the main contributors.

7.4 Comparison with Existing Approaches

Against commercial units, this hybrid approach wins on cost and accessibility, as covered in the Past and Related Work section. Against simpler standalone Raspberry Pi or ESP32-CAM builds, it adds web-based management and client-side processing. It depends on the server PC's processing power, but it keeps the core matching local and uses plain HTTP for hardware control, which lowers both cost and the privacy exposure of sending facial data off-device compared with a pure cloud API. Logging both successful and failed attempts gives it an auditing feature that simple prototypes often leave out.

8. Challenges Faced

Building the prototype ran into a number of problems that needed debugging, research, and some changes to the original plan.

Team coordination and procurement were not issues here. The project was done alone, so there were no scheduling or cooperation problems between team members, and the core parts (ESP32, LEDs, resistors, webcam, PC) were standard hardware that was easy to obtain.

Initial face detection reliability.

The first approach sent the full webcam frame from the browser to Flask for detection and recognition. This was unreliable because of variation in lighting and face position and the load it put on the server. The fix was to move detection into the browser with `face-api.js`: JavaScript guides the user to centre their face in an oval and crops the detected face before sending the smaller, centred image to the backend. This made the backend's embedding generation and comparison noticeably more reliable and faster.

Firebase confusion.

Early on it was easy to mix up Firebase Authentication, which holds login credentials, with the Realtime Database, which stores application data like embeddings and logs. This caused errors when trying to re-register users whose accounts still existed in Authentication but had been removed from the database. Once the two roles were clear, deletion was handled in the Authentication console rather than only the database, and the registration logic in `app.py` was changed to rely on Firebase Authentication's own duplicate-email handling instead of a less reliable manual check.

Client-side scripting (`face-api.js`).

A lot of time went into JavaScript debugging. The models sometimes failed to load because of connectivity or firewall issues, which stopped detection entirely. Mismatched versions of the library and the pre-trained models caused detection failures and runtime errors such as `TypeError: Cannot read properties of undefined (reading 'right')`. Race conditions also came up where the detection loop started before the camera stream or the models were ready.

The fixes were to confirm connectivity and that nothing was blocking `cdn.jsdelivr.net`, to align the script on the `@vladmandic/face-api` version that matched the models, to use Promises and `async/await` in `init()` so the camera and models were ready before `runDetection` started, and to use the correct `faceapi.resizeResults` function to clear the `TypeError`.

Flask-to-browser communication.

The browser sometimes received an HTML error page from Flask when it was expecting JSON, which broke JavaScript parsing with `SyntaxError: Unexpected token '<'`. Wrapping the `/capture` and `/verify` routes in `try` and `except` blocks and always returning valid JSON through `jsonify()`, even on an internal error, fixed this, along with checking the response `Content-Type` in the `fetch` handler before parsing.

ESP32 Wi-Fi connectivity.

Getting a stable connection and reading the assigned IP address took some care with the SSID and password and the Serial Monitor. Adding timeouts and retry logic to the Arduino Wi-Fi code, and reading the IP from the Serial Monitor at 115200 baud after a reset, made this dependable.

Flask-to-ESP32 reliability.

Occasionally the HTTP request from Flask to the ESP32 would time out if the board dropped off the network. Adding a timeout (`timeout=3`) to the `requests.get()` call and catching network errors with `try` and `except` `RequestException` let Flask keep responding to the browser even when a command to the ESP32 failed, and tightening the loop handling on the ESP32 improved stability.

Verification tolerance tuning.

The default tolerance of 0.6 was too lenient and allowed false acceptances. Adjusting it experimentally in the `compare_faces()` call settled on 0.5, which removed false acceptances while keeping false rejections for the registered user low enough for a prototype.

Camera access conflicts.

The browser sometimes reported the webcam as already in use. This usually meant another application (Zoom, Teams, another tab, or a leftover Python script) was holding the camera. Adding cleanup logic in JavaScript (`stopCameraStream()` and a `beforeunload` listener) to release the camera on page close helped, and the reliable fix is simply to make sure nothing else is using the camera before starting the app.

9. Final Budget

The full bill of materials for the prototype is listed below. Most parts were inexpensive and bought locally.

Component	Qty	Unit (ZAR)	Total (ZAR)	Notes	Shop
ESP32 WROOM dev board	1	R 115	R 115	Core microcontroller	Communica
Green LED (5 mm)	1	R 2	R 2	Access-granted indicator	Botshop
Red LED (5 mm)	1	R 2	R 2	Access-denied / idle indicator	Botshop
Resistors (220 Ohm, 1 kOhm pack)	1 pack	R 105	R 105	220 Ohm for LEDs, 1 kOhm for relay input	Communica
Jumper wires (assorted)	1 pack	R 40	R 40	Connecting components	Botshop
Micro-USB cable	1	R 100	R 100	Power and programming for the ESP32	Cell-fix
5 V relay module	1	R 25	R 25	Switches the solenoid current	Botshop
12 V solenoid lock	1	R 169.53	R 169.53	Locking mechanism	Botshop
12 V power supply (1 A+)	1	R 230	R 230	Powers the solenoid	Communica
TIP41C transistor	1	R 7.99	R 7.99	Drives the relay	Communica
100 uF capacitors	2	R 4.49	R 4.49	Only needed with the LM7805	Communica
Spray paint	2	R 102	R 102	Finishing the PVC enclosure	Easy Build
PVC pipe	1	R 80	R 80	House and door body	Easy Build
I2C LCD	1	R 99.99	R 99.99	Status display	Botshop
15 mm and 20 mm nuts and bolts	20	R 121	R 121	Assembly and stability	Communica
Subtotal (full system)		R 1204	R 1204		

10. Project Deliverables

The project set out to design, build, and test a prototype facial recognition access control system that combines web technologies with IoT hardware. The deliverables are:

Functional prototype

- A working demonstration: the Flask web application on a host PC, a standard webcam, and an ESP32 connected over Wi-Fi.
- Indicator LEDs (red and green) and the solenoid lock driven by the ESP32 according to the verification result.

Software codebase

- **Flask application (app.py):** the backend logic for registration, login through Firebase Auth, face capture handling, embedding generation and comparison, data storage and retrieval in the Realtime Database through Pyrebase, access logging, and ESP32 command transmission.
- **Frontend templates:** five HTML files (register, login, capture, verify, dashboard) styled with Tailwind CSS, with embedded JavaScript for webcam interaction, client-side detection and cropping (face-api.js), and communication with the backend (fetch).
- **ESP32 firmware (.ino):** the Arduino sketch that handles Wi-Fi, runs a web server, and controls the GPIO pins for the LEDs and the relay based on received HTTP commands.

Configuration

- `firebaseConfig.json`: the API keys and URLs that connect the Flask application to the Firebase project.

Hardware assembly

- The physical circuit on Veroboard connecting the ESP32, the LEDs and resistors, and the relay that opens the solenoid lock, mounted in the PVC enclosure.

Final report

- This document, covering the background, objectives, design, implementation, management, results, challenges, costs, and conclusions.

10.1 Comparison with Initial Goals

The project met its main goal: a working prototype that combines facial recognition through a web application with hardware feedback driven by an ESP32. Every core function from the original concept was implemented and demonstrated, including registration, face capture, verification, hardware signalling through the LEDs and the solenoid lock, and access logging. Firebase handles authentication and data storage as planned.

The honest limitations are about how well it holds up in varied conditions, not about missing features. Recognition is sensitive to lighting and face angle because it compares against a single registration photo, and the prototype has no liveness detection or 3D sensing, so advanced anti-spoofing was out of scope. These are the natural next steps rather than gaps in the core functionality.

11. Recommendations and Future Work

A few clear improvements would take the prototype closer to something deployable.

Liveness detection.

As it stands, the system could be fooled by a photo or video of the registered user. Adding liveness detection would close that gap, either in software by looking for small movements like eye blinks or analysing texture in the live feed, or in hardware by adding an infrared or depth camera to tell a real face from a flat image. The hardware route would mean meaningful changes to both the wiring and the processing.

Recognition reliability.

Accuracy currently leans on lighting and angle matching the one registration photo. Capturing several images per user from different angles and lighting, and storing multiple embeddings or an averaged one, would make matching more forgiving. Adjusting the tolerance based on ambient light or image quality, and trying more capable recognition models, would help too, at the cost of more processing.

User feedback.

The LEDs are minimal. Distinct buzzer patterns for success, failure, and errors, a small LCD or OLED showing messages like Verifying or Access Granted, and push notifications through something like Firebase Cloud Messaging would all make the system clearer to use and easier to monitor.

Offline capability.

Right now the system depends entirely on the Flask server and Wi-Fi, so an outage leaves the lock unusable. Adding a fallback is possible but involved: storing a few embeddings on the ESP32's flash, running a lightweight model such as ESP-WHO directly on an ESP32-CAM, or keeping a local backup such as RFID.

Scalability and management.

The current build suits a single user or a handful added through registration. An admin web interface in the Flask app would let authorised staff add and remove users, view everyone's logs, manage permissions, and update embeddings without touching the code or the hardware.

Power management.

An always-on ESP32 running Wi-Fi and a web server drains a battery quickly. Deep sleep, with the board waking only when a motion sensor or doorbell triggers it before running verification, would cut power use sharply, though it adds complexity to the trigger.

12. Conclusions

This project developed and demonstrated a working facial recognition door lock that ties web technologies to IoT hardware. The aim was a secure, convenient alternative to keys and PINs using biometric verification, and the supporting goals were web-based registration and user management, capturing and storing face embeddings in Firebase, comparing faces reliably, sending the result to an ESP32 over Wi-Fi, showing that result on LEDs, and opening the solenoid lock while logging each attempt.

Those goals were largely met. Users can register, log in, capture their face, and be verified with a standard webcam and browser. The Flask backend decodes the image, compares the embedding against the one stored in Firebase using a tuned tolerance, and sends the right command to the ESP32, which drives the LEDs and the lock as intended. Both successful and failed attempts are logged with timestamps in Firebase, giving a usable audit trail.

Most of the difficulties were in integration and environment rather than the core idea. Server-side detection turned out to be unreliable, which forced a move to client-side detection and cropping with `face-api.js` and brought its own issues with script versions, model loading, and asynchronous timing. Communication between Flask and the ESP32, and the Firebase interactions, needed careful error handling on both the Python and Arduino sides. Getting the recognition tolerance right was also important for balancing security against everyday usability.

What makes the work useful is that it shows a hybrid design built from cheap, easily available parts: a standard PC and webcam, open-source libraries, Firebase's free tier, and a low-cost ESP32, working together as one access control system. As a proof of concept it holds up, and the features it does include, client-side processing, an adjustable tolerance, and separate logging of successful and failed attempts, make it a reasonable starting point for a more complete physical security product.

13. References

- [1] Pallets Projects, "Flask Documentation," 2024. [Online]. Available: <https://flask.palletsprojects.com/>. [Accessed: 25 Oct. 2025].
- [2] A. Geitgey, "face_recognition 1.3.0," Python Package Index (PyPI), 2018. [Online]. Available: <https://pypi.org/project/face-recognition/>. [Accessed: 25 Oct. 2025].
- [3] Pyrebase, "A simple Python wrapper for the Firebase API," GitHub. [Online]. Available: <https://github.com/thisbewhatdev/pyrebase>. [Accessed: 25 Oct. 2025].
- [4] Google, "Firebase Documentation." [Online]. Available: <https://firebase.google.com/docs>. [Accessed: 25 Oct. 2025].
- [5] V. Mandic, "face-api.js," GitHub. [Online]. Available: <https://github.com/vladmandic/face-api>. [Accessed: 25 Oct. 2025].
- [6] Arduino, "Arduino Official Website." [Online]. Available: <https://www.arduino.cc/>. [Accessed: 25 Oct. 2025].
- [7] Espressif Systems, "ESP32 Series Datasheet." [Online]. Available: <https://www.espressif.com/en/support/documents/technical-documents>. [Accessed: 25 Oct. 2025].
- [8] Python Software Foundation, "Welcome to Python.org." [Online]. Available: <https://www.python.org/>. [Accessed: 25 Oct. 2025].
- [9] Requests, "Requests: HTTP for Humans." [Online]. Available: <https://requests.readthedocs.io/en/latest/>. [Accessed: 25 Oct. 2025].
- [10] A. Wathan et al., "Tailwind CSS." [Online]. Available: <https://tailwindcss.com/>. [Accessed: 25 Oct. 2025].
- [11] F. Lundh, "Pillow (Fork of PIL)," Python Package Index (PyPI). [Online]. Available: <https://pypi.org/project/Pillow/>. [Accessed: 25 Oct. 2025].

14. Appendix A: Project Progression Images

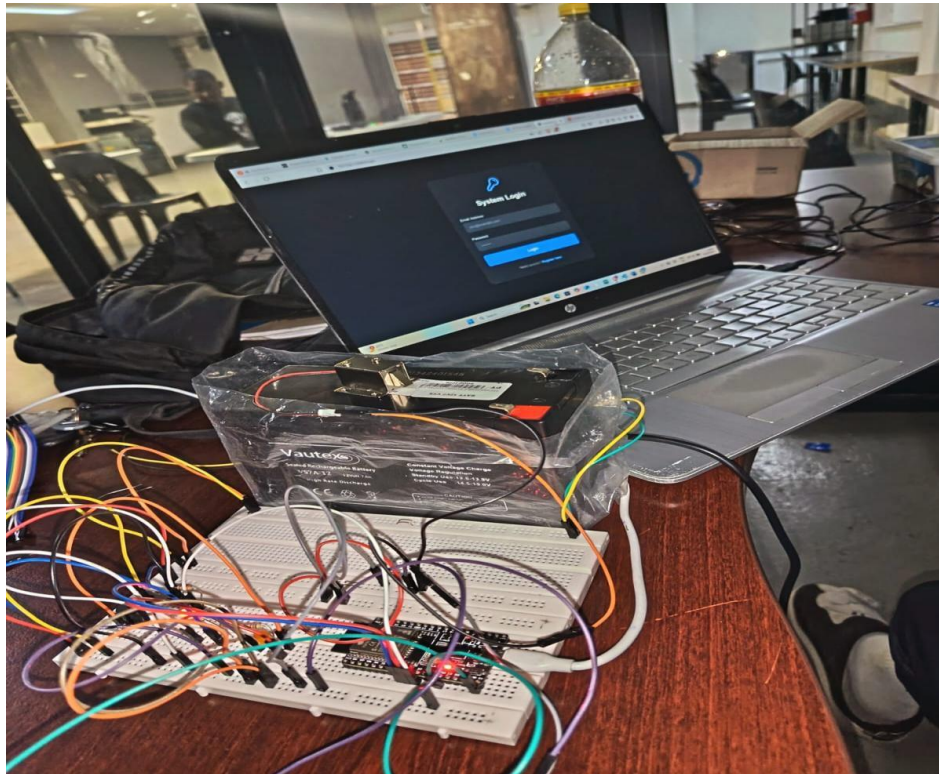


Figure A.1. Breadboard prototype with the login page open during testing.

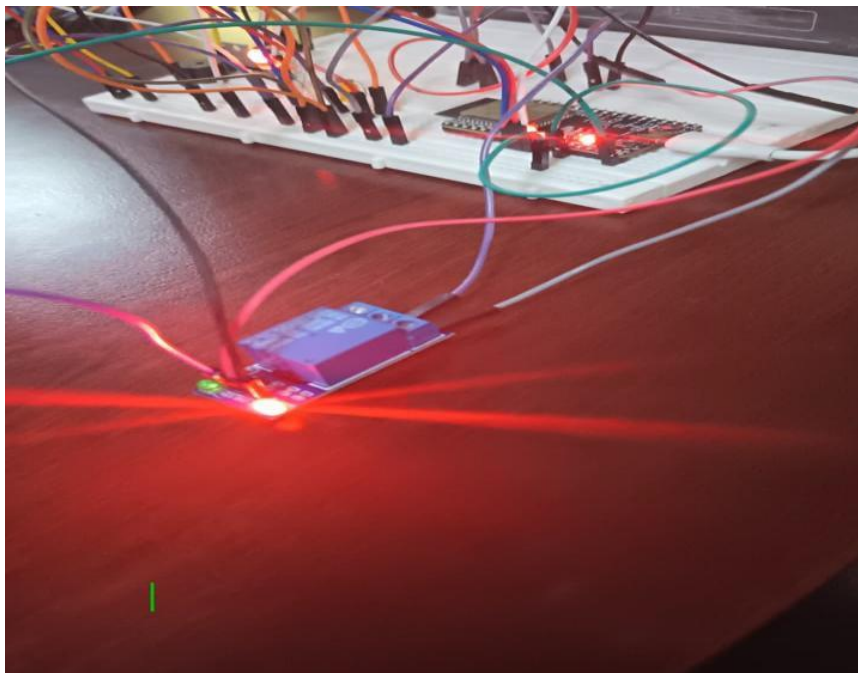


Figure A.2. Relay module switching, with the red indicator active.

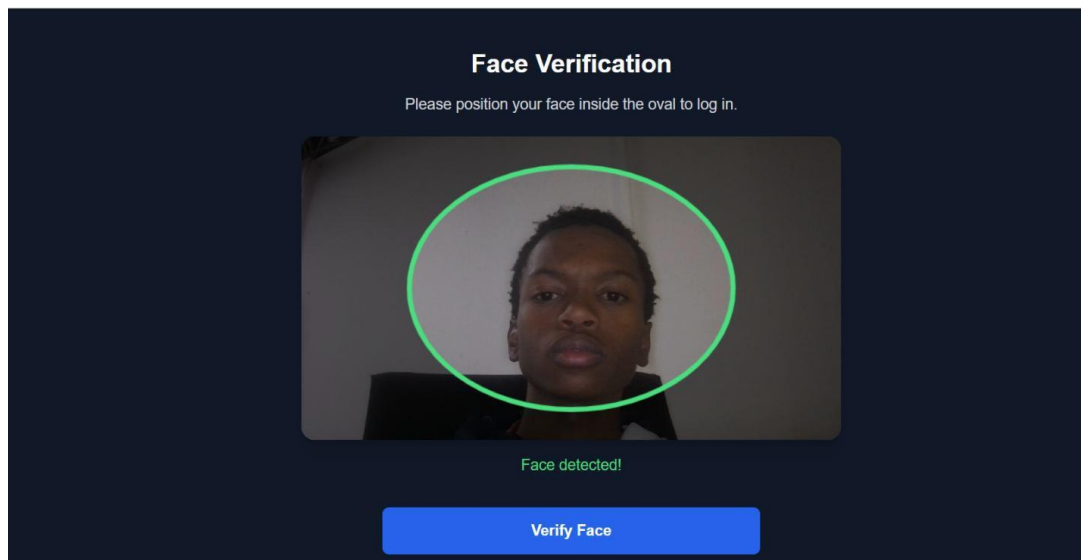


Figure A.3. Face verification page detecting a centred face in the browser.



Figure A.4. Final prototype enclosure with the LCD and solenoid lock.

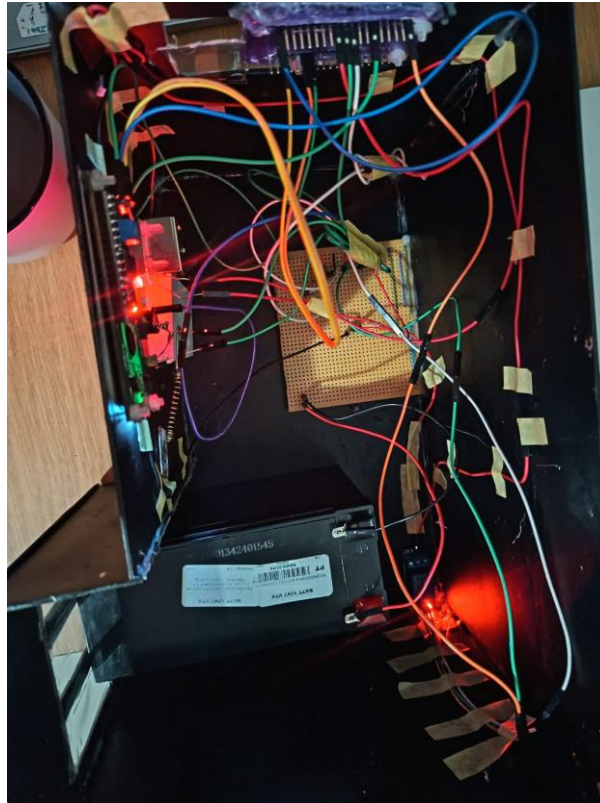


Figure A.5. Internal wiring inside the enclosure.

15. Appendix B: Source Code

The key excerpts of each source file are shown below. The full project files (the complete app.py, the ESP32 sketch, and the five HTML templates) are submitted alongside this report.

Flask backend (app.py)

```
import json, os, pyrebase, base64, io
import numpy as np
from flask import Flask, request, jsonify, session, render_template

# ... routes: /register, /login, /capture, /verify, /dashboard ...
# ... face embedding generation and compare_faces() with tolerance=0.5 ...
# ... Firebase Realtime Database read/write and access logging ...
# ... requests.get() to the ESP32 for /access_granted or /access_denied ...

if __name__ == '__main__':
    print('Starting Flask app...')
    app.run(debug=True, host='0.0.0.0', port=5000)
```

ESP32 firmware (.ino)

```
#include <WiFi.h>
#include <WebServer.h>

// --- Network credentials ---
const char* ssid      = "YOUR_WIFI_SSID";
const char* password = "YOUR_WIFI_PASSWORD";

// --- Pin definitions ---
const int greenLedPin = 26;
const int redLedPin   = 25;
const int relayPin    = 27;

// --- Configuration ---
const int unlockDuration = 2000; // ms

WebServer server(80);

void setup() {
    Serial.begin(115200);
    // ... pin modes, Wi-Fi connect, route handlers ...
}

void loop() {
    server.handleClient();
    delay(10);
}

// connectWiFi(), handleAccessGranted(), handleAccessDenied(),
// handleNotFound(), updateActuators()
```

Frontend templates (templates/)

Five HTML templates make up the interface: register.html, login.html, capture.html, verify.html, and dashboard.html. Each is styled with Tailwind CSS and uses embedded JavaScript for webcam access, client-side face detection and cropping with face-api.js, and fetch calls to the Flask backend.